

Intermediate (build real memory) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — the direct way

```
history = [
    {"role": "system", "content": "You are a helpful assistant."}
]

while True:
    user_input = input("You: ")
    if user_input == "quit":
        break

    history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    reply = response.choices[0].message.content

    history.append({"role": "assistant", "content": reply})
    print("Assistant:", reply)
```

Expected output (a real run):

```
You: My favorite color is teal.
Assistant: Teal is a great choice! It's a lovely blend of blue and green...
You: What's my favorite color?
Assistant: You mentioned your favorite color is teal!
```

This works correctly and proves memory. It has no error handling, and it never limits how large `history` can grow, which matters once conversations get long.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `get_reply()` pulled out, with type hints

```
def get_reply(history: list) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content

def run_chat() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."}
    ]

    while True:
        user_input = input("You: ")
        if user_input == "quit":
            break

        history.append({"role": "user", "content": user_input})

        reply = get_reply(history)
        history.append({"role": "assistant", "content": reply})

        print("Assistant:", reply)

if __name__ == "__main__":
    run_chat()
```

Expected output: identical conversation to Basic — same behavior, cleaner structure.

Difference from Basic: pulling the API call out into `get_reply(history)` separates "how to talk to the model" from "how the loop is structured" — this is the exact shape the Build Task's `send_message()` needs to take. Full type hints on `get_reply` document that it takes the whole history list and returns just the new reply text, not a whole response object the caller has to unpack.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a `ChatSession` class, so history belongs to an instance

```
from openai import OpenAI

class ChatSession:
    def __init__(self, client: OpenAI, system_prompt: str) -> None:
        self.client = client
        self.history: list[dict] = [{"role": "system", "content": system_prompt}]

    def send(self, user_input: str) -> str:
        self.history.append({"role": "user", "content": user_input})
        response = self.client.chat.completions.create(
            model="gpt-4o-mini",
            messages=self.history,
        )
        reply = response.choices[0].message.content
        self.history.append({"role": "assistant", "content": reply})
        return reply

client = OpenAI()
session_a = ChatSession(client, "You are a helpful assistant.")
session_b = ChatSession(client, "You are a helpful assistant.")

print(session_a.send("My name is Priya."))
print(session_b.send("What is my name?"))
```

Expected output:

```
Nice to meet you, Priya! How can I help you today?
I don't have access to that information – could you tell me your name?
```

`session_b` genuinely has no idea, because its `history` never saw `session_a`'s messages — proving the two conversations are actually independent, not one shared list two callers happen to both be looking at.

Approach 2 — the same class, plus a simple length cap on `history`

Answering the second Advanced question from Hint 1 — what happens once `history` gets too long — with the simplest possible real answer: once it passes a threshold, drop the oldest non-system turns, keeping the conversation going instead of growing forever toward the context-limit error you'll see on purpose in this document's Edge cases exercise.

```
from openai import OpenAI

class ChatSession:
    def __init__(self, client: OpenAI, system_prompt: str, max_turns: int = 20) -
        self.client = client
        self.history: list[dict] = [{"role": "system", "content": system_prompt}]
        self.max_turns = max_turns

    def send(self, user_input: str) -> str:
        self.history.append({"role": "user", "content": user_input})
        response = self.client.chat.completions.create(
            model="gpt-4o-mini",
            messages=self.history,
        )
        reply = response.choices[0].message.content
        self.history.append({"role": "assistant", "content": reply})
        self._trim_if_needed()
        return reply

    def _trim_if_needed(self) -> None:
        # +1 for the system message, which always stays at index 0
        max_entries = (self.max_turns * 2) + 1
        if len(self.history) > max_entries:
            system_message = self.history[0]
            recent = self.history[-(max_entries - 1):]
            self.history = [system_message] + recent
```

Expected output after many turns past `max_turns` : the conversation keeps working, but the model can no longer answer questions about the very first few turns — a deliberate, visible trade-off, not a silent crash.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `get_reply()` / `run_chat()` pair only works for exactly one conversation at a time, held together by loose module-level variables. Approach 1 fixes the multi-conversation problem by moving `history` inside a class, so each `ChatSession` owns its own list. Approach 2 keeps that same structure and adds the second Advanced concern — an unbounded `history` — trimming automatically once it passes `max_turns`, instead of leaving that problem for the context-limit error to eventually cause.

Which one should you actually write? For a single-user terminal script — which is exactly what this document's Basic and Real-world exercises are — Intermediate's plain `history` list is completely fine, and simpler to read. Reach for Advanced Approach 1's `ChatSession` class the moment more than one conversation needs to exist at the same time, which is true starting with this document's own Build Task (a chatbot that should work correctly no matter how many times you restart it or how it's reused later). Approach 2's trimming is worth adding once you've actually seen a conversation grow long enough to worry about — pairing it with the `tiktoken`-based proactive check from the Edge cases exercise gives you both a soft warning and a hard safety net.